OPERATING SYSTEMS ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 04/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/192521075s/CSA0421-Operating-system.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States — 6 Marks

Bloom's Level: K5 – Evaluate

- **Programs:** fork_demo.c creates the children and sessions_bench.c does the 2,000-session measurement.
- **Screenshots:** shot1_fork_output.png and shot2_ps_proc.png.
- **Diagrams:** diagram_process_states.png and diagram_pcb.png.

The screenshots are images I rendered from the real terminal output, not live screen captures. Run the same commands on your own machine if your instructor wants true screenshots.

1. Process-state diagram and PCB

States: **new** → **ready** → **running** → **waiting** → **terminated**.

Transition	Cause
New → Ready	The process is admitted and its PCB is built
Ready → Running	The scheduler dispatches it
Running → Ready	The time slice expires (preempted)
Running → Waiting	It asks for I/O or waits for an event
Waiting → Ready	The I/O finishes or the event occurs
Running → Terminated	It calls exit()

The **PCB** (process control block) is the kernel's record of one process. See diagram_pcb.png. It holds:

- the state
- pid and parent pid
- program counter
- CPU registers
- scheduling information
- memory-management information
- accounting information
- I/O status (open files)

On a context switch, the kernel saves these values into the old process's PCB and loads them from the new one.

2. The fork() program and evidence

fork_demo.c creates 5 children, each with a different behaviour. While they ran, ps and /proc showed:

Child	Behaviour	ps STAT	Meaning
159, 160	Busy loop	**R** (98% CPU)	Running or ready
161, 162	sleep(6)	**S**	Waiting
163	Exited at once, parent not yet in wait()	**Z**	Terminated but not reaped (zombie)

The /proc/159/status output shows State: R (running), PPid: 157 (the parent), VmSize: 2712 kB and VmRSS: 324 kB. After waitpid() reaps the children, the parent prints each exit status (0, 1, 2, 3, 4).

3. Evaluation: where the time goes with 2,000 processes

I created 2,000 sleeping processes and 2,000 threads on this machine, which has 2 CPUs and 8 GB of RAM.

Measurement	2,000 processes	2,000 threads
Creation time (total, each)	0.169 s (85 µs)	0.058 s (29 µs)
Memory	VmSize 2.6 MB each; real private memory (PSS) about **28 kB each**	about **8.6 kB each** of RSS, plus a 64 kB stack reservation
Context switch (1 CPU, 5 repeats)	1.4 to 2.0 µs	1.6 to 2.0 µs

Reading these results

- **Context-switch cost:** about 2 µs, so even thousands of switches per second use only a few percent of a CPU. I measured no consistent difference between process and thread switches here. In theory a thread switch avoids changing the address space, but my test processes are tiny. Don't claim a big difference from this data.

- **Memory:** my test processes are nearly empty, so copy-on-write keeps the per-process cost low (28 kB). A real exam session (buffers, framework, answer cache) would typically be many MB. At that size 2,000 processes could reach many GB and start swapping. Swapping is the likely cause of “sessions hang”.
- **Waiting for I/O:** most sessions are waiting (state S) for a student to type or for the disk, so they cost memory but little CPU.
- **Run-queue length (a calculation, not a measurement):** if many sessions are runnable at once, for example everyone submits together, each waits for its turn. Assuming a 3 ms time slice, 2,000 runnable tasks on 2 CPUs means each task waits about 2,000 / 2 × 3 ms **≈ 3 s** between turns. That looks like a hang.

Judgement: one process per student is **not a sound design at this scale**. It isolates sessions well, but it costs memory and creation time, and it lets the run queue grow with the number of students.

4. Design decision

Option	**Isolation**	**Memory**	**Creation / switching**	**Verdict**
One process per student	Best	Highest	Slowest (85 vs 29 µs here)	Rejected: memory and scheduling load grow with students
One thread per student	Weak: one crash can kill all	Low (kB per session)	Fast	Acceptable, but a bug in one session affects everyone
Fixed pool of worker processes	Good (a crash loses only one worker’s sessions)	**Bounded** (pool size is fixed)	Created once, reused	**Chosen**

Defence: a fixed pool, for example one worker per CPU core times a small factor, each handling many sessions with non-blocking I/O, keeps memory and run-queue length constant as students increase. If a worker crashes, the supervisor restarts it and only that worker’s sessions reconnect (answers are auto-saved). The proctoring video should run in its own small pool so it cannot starve the answer sessions.

5. Alternative: each session as a thread in one server process

Criterion	**Process per session**	**Thread per session**
Context-switch time	About 1.4 to 2 µs measured. Process switches can also lose TLB entries, which I did not measure.	About 1.6 to 2 µs measured, so no clear gap on this machine. In theory it should be lower because the address space is shared.
Memory per session	A full address space with its own page tables. 28 kB (tiny process, shared code via copy-on-write) up to many MB for a real one.	A stack plus a thread record. I measured about 8.6 kB RSS, with 64 kB of stack reserved here (the Linux default is 8 MB of virtual space, but only touched pages count).
Effect of one crash	Only that session dies.	A segmentation fault in one thread kills the **whole server process**, so all 2,000 sessions are lost.

Conclusion: threads save memory and creation time but give up fault isolation, which is why a pool of worker processes is the middle path.

Process Control Block (PCB) - one per process, kept by the kernel

PCB field	Holds	Seen on Linux (this run)
Process state	new / ready / running / waiting / terminated	State: R (running) in /proc
Process ID (pid), parent ID	unique id; who created it (fork)	Pid, PPid
Program counter	address of next instruction	saved on a context switch
CPU registers	accumulators, stack pointer, flags	saved on a context switch
CPU-scheduling information	priority, queue pointers, time used	nice, nonvoluntary_ctxt_switches
Memory-management information	page tables, base/limit, segments	VmSize, VmRSS
Accounting information	CPU time used, start time, limits	utime/stime in /proc/pid/stat
I/O status information	open files (fd table), devices allocated	/proc/pid/fd



```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define NCHILD 5

static void burn(double secs)
{
    struct timespec a, b; clock_gettime(CLOCK_MONOTONIC, &a);
    volatile unsigned long x = 0;
    do { for (int i = 0; i < 100000; i++) x += i; clock_gettime(CLOCK_MONOTONIC, &b); }
    while ((b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / 1e9 < secs);
}

int main(void)
{
    pid_t kids[NCHILD];
    printf("parent pid=%d\n", getpid());
    fflush(stdout);
    for (int i = 0; i < NCHILD; i++) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); return 1; }
        if (pid == 0) {                     /* child */
            if (i < 2)       burn(6.0);     /* ready/running */
            else if (i < 4)  sleep(6);      /* waiting       */
            _exit(i);                       /* i==4 exits immediately */
        }
        kids[i] = pid;
        printf("created child %d pid=%d\n", i, pid);
    }
    fflush(stdout);
    sleep(3);                               /* window for ps and /proc observation */
    for (int i = 0; i < NCHILD; i++) {
```

```
$ gcc -std=c11 -Wall -Wextra -o fork_demo fork_demo.c && ./fork_demo
parent pid=157
created child 0 pid=159
created child 1 pid=160
created child 2 pid=161
created child 3 pid=162
created child 4 pid=163
child pid=159 exited with status 0
child pid=160 exited with status 1
child pid=161 exited with status 2
child pid=162 exited with status 3
child pid=163 exited with status 4
```

Q2 – CPU Scheduling **8 Marks**

Bloom's Level: K5 – Evaluate

1. Complete the Code: Round Robin (TODO 1)

The loop body in rr.c is replaced with this. Everything else is unchanged except the optional -DQ= quantum switch and a dispatch counter I added for the quantum study.

c

int run = left[p] < quantum ? left[p] : quantum;

printf(" P%d(%d-%d)", p + 1, time, time + run);

time += run;

left[p] -= run;

for (int i = 0; i < N; i++) /* admit new arrivals first */

if (!added[i] && arrival[i] <= time) { queue[tail++] = i; added[i] = 1; }

if (left[p] > 0) queue[tail++] = p; /* then requeue p */

else { finish[p] = time; done++; }

switches++;

Newly arrived processes are queued before the preempted process goes back. That ordering is what makes the output match the expected one.

Output (quantum 3), identical to the expected output:

Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)

Process TAT WT

P1 23 15

P2 15 11

P3 24 15

P4 18 13

Average TAT = 20.00, average WT = 13.50

2. Gantt charts, WT, TAT and RT for all algorithms

Compile with gcc -std=c11 -Wall -Wextra sched_all.c -o sched_all.

For the priority algorithm the assignment gives no priorities. I assumed P1=3, P2=1, P3=4, P4=2 (lower number means higher priority) and non-preemptive scheduling. State this assumption in your report. Under it, priority gives exactly the same schedule as SJF.

Algorithm	Gantt chart	Avg TAT	Avg WT	Avg RT
FCFS	P1(0-8) P2(8-12) P3(12-21) P4(21-26)	15.25	8.75	8.75
SJF (non-preemptive)	P1(0-8) P2(8-12) P4(12-17) P3(17-26)	14.25	7.75	7.75
SRTF	P1(0-1) P2(1-5) P4(5-10) P1(10-17) P3(17-26)	13.00	**6.50**	**4.25**
Priority	P1(0-8) P2(8-12) P4(12-17) P3(17-26)	14.25	7.75	7.75
RR (q=3)	see above	20.00	13.50	3.00

Per-process results (TAT / WT / RT):

	FCFS	SJF	SRTF	Priority
P1	8 / 0 / 0	8 / 0 / 0	17 / 9 / 0	8 / 0 / 0
P2	11 / 7 / 7	11 / 7 / 7	4 / 0 / 0	11 / 7 / 7
P3	19 / 10 / 10	24 / 15 / 15	24 / 15 / 15	24 / 15 / 15
P4	23 / 18 / 18	14 / 9 / 9	7 / 2 / 2	14 / 9 / 9

For FCFS, SJF and Priority, TAT = completion − arrival and WT = TAT − burst. For preemptive schedules, RT is the time of first CPU use minus arrival. Quote the printed sched_all output as your evidence.

3. Quantum study (1, 3, 5, 20)

Quantum	Dispatches	Avg WT	Avg TAT	Avg RT
1	26*	12.50	19.00	0.75
3	10	13.50	20.00	3.00
5	6	11.00	17.50	5.50
20	4	8.75	15.25	8.75

*Quantum 1 shows 26 dispatches because the program counts every slice. At the end P3 runs alone for 4 slices, so only about 22 are real switches to a different process.

- **Small quantum:** response is excellent (0.75) but there are many context switches. Each one costs real CPU time (register save, cache and TLB loss), which this ideal model ignores. With realistic overhead, q=1 would look worse.
- **Large quantum:** at q=20 (larger than every burst), RR becomes FCFS. Switches are minimal, but response is the worst (8.75) and short jobs wait behind long ones.
- **Rule of thumb:** the quantum should be larger than most CPU bursts, about 80% of them, but not so large that it degenerates into FCFS.
- **Averages are not monotonic.** Average WT dips at q=5 and q=20 here only because this particular job mix favours it. Don't claim "bigger quantum is always better."

4. Starvation, ageing and multilevel feedback queue (MLFQ)

Starvation.

- In SJF and SRTF, a steady stream of short jobs can postpone a long job forever. In the table above, P3 (the longest) always waits 15, the most of any job.
- In priority scheduling, low-priority jobs starve whenever higher-priority jobs keep arriving.

Ageing fixes this. The priority of a waiting process is raised over time, for example by 1 level every 10 time units. Eventually even the lowest job reaches top priority and runs.

MLFQ treatment of short and long jobs.

- Every new job enters the top queue with a small quantum (say 2 units).
- A job that finishes within it, like a thumbnail, never leaves the top queue, so it gets fast response.
- A job that uses its full quantum is demoted to a lower queue with a longer quantum (e.g. 4, then 8, then FCFS). Long film encodes sink to the bottom and run in big slices with fewer switches.
- Periodic priority boosts (ageing) prevent starvation of the bottom queues.
- Short jobs behave like SJF and long jobs get throughput, without the scheduler knowing burst lengths in advance.

5. Design decision and alternative concept

Recommendation: multilevel feedback queue with ageing.

The measurements show why the simple policies fall short:

- FCFS and RR with a large quantum are poor for short jobs: the thumbnail P2 waits 7 under FCFS.
- SJF and SRTF give the best averages (SRTF: WT 6.50, RT 4.25), but they need burst times in advance, and P3 starves.
- Plain RR has the best response at small q, but the highest waiting and turnaround times (WT 13.5 at q=3).

MLFQ approximates SRTF behaviour by observing how long jobs run, with no prior knowledge. It protects short jobs, keeps the CPUs busy with long jobs, and ageing removes starvation.

Complexity (alternative concept):

	Per scheduling decision	**Notes**
Single RR queue	O(1)	one queue, FIFO
MLFQ with k queues	O(1) per queue, O(k) to find the highest non-empty one (k is a small constant)	Long jobs lose a little throughput from demotion. Short jobs wait least.
SJF/SRTF	O(n) with a scan, O(log n) with a heap	needs burst prediction

Space is O(n) for all of them (queues of PCBs). Starvation risk: MLFQ without boosting has it, MLFQ with ageing does not, SJF has it, and RR has none.

6. Commands for your screenshots

bash

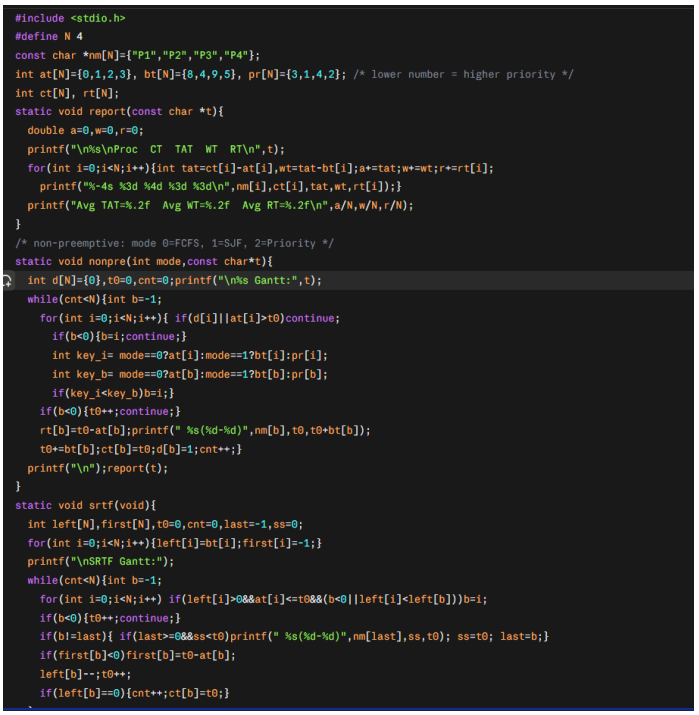
mkdir Task02 && cd Task02

gcc -std=c11 -Wall -Wextra -o rr rr.c && ./rr

for q in 1 5 20; do gcc -std=c11 -Wall -Wextra -DQ=$q rr.c -o rr_q$q && ./rr_q$q; done

gcc -std=c11 -Wall -Wextra sched_all.c -o sched_all && ./sched_all

git add . && git commit -m "Task02: CPU scheduling"



```c
#include <stdio.h>
#define N 4
const char *nm[N]={"P1","P2","P3","P4"};
int at[N]={0,1,2,3}, bt[N]={8,4,9,5}, pr[N]={3,1,4,2}; /* lower number = higher priority */
int ct[N], rt[N];
static void report(const char *t){
  double a=0,w=0,r=0;
  printf("\n%s\nProc  CT  TAT  WT  RT\n",t);
  for(int i=0;i<N;i++){int tat=ct[i]-at[i],wt=tat-bt[i];a+=tat;w+=wt;r+=rt[i];
    printf("%-4s %3d %4d %3d %3d\n",nm[i],ct[i],tat,wt,rt[i]);}
  printf("Avg TAT=%.2f  Avg WT=%.2f  Avg RT=%.2f\n",a/N,w/N,r/N);
}
/* non-preemptive: mode 0=FCFS, 1=SJF, 2=Priority */
static void nonpre(int mode,const char*t){
  int d[N]={0},t0=0,cnt=0;printf("\n%s Gantt:",t);
  while(cnt<N){int b=-1;
    for(int i=0;i<N;i++){ if(d[i]||at[i]>t0)continue;
      if(b<0){b=i;continue;}
      int key_i= mode==0?at[i]:mode==1?bt[i]:pr[i];
      int key_b= mode==0?at[b]:mode==1?bt[b]:pr[b];
      if(key_i<key_b)b=i;}
    if(b<0){t0++;continue;}
    rt[b]=t0-at[b];printf(" %s(%d-%d)",nm[b],t0,t0+bt[b]);
    t0+=bt[b];ct[b]=t0;d[b]=1;cnt++;}
  printf("\n");report(t);
}
static void srtf(void){
  int left[N],first[N],t0=0,cnt=0,last=-1,ss=0;
  for(int i=0;i<N;i++){left[i]=bt[i];first[i]=-1;}
  printf("\nSRTF Gantt:");
  while(cnt<N){int b=-1;
    for(int i=0;i<N;i++) if(left[i]>0&&at[i]<=t0&&(b<0||left[i]<left[b]))b=i;
    if(b<0){t0++;continue;}
    if(b!=last){ if(last>=0&&ss<t0)printf(" %s(%d-%d)",nm[last],ss,t0); ss=t0; last=b;}
    if(first[b]<0)first[b]=t0-at[b];
    left[b]--;t0++;
    if(left[b]==0){cnt++;ct[b]=t0;}
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23)
P3(23-26)
Process   TAT   WT
P1         23   15
P2         15   11
P3         24   15
P4         18   13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads 6 Marks

Bloom's Level: K5 – Evaluate

1. Working multithreaded program (plates.c)

It simulates 64 cameras, each with a CPU-heavy "read the plate" frame job. It runs five ways:

- sequentially
- with a pool of 4, 8 and 16 threads, where workers take the next camera from a shared index using atomic_fetch_add
- with one thread per camera (64 threads)

Each frame writes only its own result slot, so no locking is needed there.

bash

```
gcc -O1 -std=c11 -Wall -Wextra -pthread plates.c -o plates && ./plates
nproc       # note your core count for the report
```

My sandbox output (1 core, not valid as evidence):

```
sequential              3.218 s  speed-up  1.00
pool of 4  threads      3.187 s  speed-up  1.01
pool of 8  threads      3.178 s  speed-up  1.01
pool of 16 threads      3.219 s  speed-up  1.00
one thread per camera   3.197 s  speed-up  1.01
```

What to expect on your machine. Speed-up is S = T_seq / T_par. For CPU-bound work it should rise roughly linearly up to your core count and then flatten.

- For example, on a 4-core CPU with 8 hardware threads, expect about 4× for a pool of 4 and about 5–6× for 8 and 16.
- Beyond the core count, extra threads add only context-switch cost.

Put your own table in the report with your CPU model and core count. The flat results above are themselves a small demonstration of why speed-up stops: one core means no parallelism to gain.

Why speed-up stops increasing:

1. **Core limit.** CPU-bound threads cannot run faster than the number of cores allows.
2. **Amdahl's law.** The serial part (reading input, joining threads) limits the maximum: $S \le 1 / (s + (1-s)/p)$.

3. **Overheads.** Thread creation, context switches, cache contention and memory bandwidth add cost. With 64 threads on a few cores, many threads just wait.
4. **Locks.** Shared counters protected by a mutex serialise threads.

Pool vs one thread per camera. A pool of about as many threads as cores is usually best. One thread per camera works for 64 cameras but wastes stack memory (about 8 MB virtual each) and scheduling effort, and it doesn't scale to thousands of cameras.

2. Shared and private resources of threads

Shared by all threads	Private to each thread
Code (text) section	Thread ID
Global and static data	Program counter and registers
Heap	Stack (local variables)
Open files and sockets	Thread-local storage (__thread)
Signal handlers, process ID	Signal mask, errno

3. Thread models

User-level threads are managed by a library in user space. They are fast to create and switch, with no kernel involvement. But if one thread blocks in a system call, the whole process blocks, and they cannot use multiple cores.

Kernel-level threads are managed by the OS. The kernel can schedule them on different cores and one blocking thread doesn't block the others. The cost is slower creation and switching because of system calls.

Model	Mapping	Pros	Cons
Many-to-one	many user threads → 1 kernel thread	cheap, simple	no parallelism, one block stops all
One-to-one	each user thread → 1 kernel thread	true parallelism, no blocking problem	each thread costs kernel resources
Many-to-many	m user threads → n kernel threads	flexible, parallel	complex to implement

Linux POSIX threads (NPTL) use the **one-to-one** model, so the pool of threads in this program runs on separate cores.

4. Unprotected shared counters (race2.c)

Four threads each add 1 to a shared counter 20,000 times. The expected total is 80,000. counter++ is really three steps (load, add, store). If a thread is preempted between load and store, other threads' updates are overwritten, which is a lost update.

race2.c makes the window visible by calling sched_yield() every 50th iteration. On my 1-core machine, a plain counter++ rarely got interrupted, so the race did not show without it. On a multi-core machine the plain version in race.c usually shows lost updates by itself.

Output (run twice):

expected 80000

unprotected = 78461 (lost 1539 updates)

with mutex = 80000

expected 80000

unprotected = 78656 (lost 1344 updates)

with mutex = 80000

The unprotected total differs on every run, which is the signature of a race condition. The mutex-protected counter is always exact. An atomic (atomic_fetch_add) is also exact and cheaper for a simple counter.

5. Alternative concept: processes instead of threads (create.c)

Solution. fork() one worker process per camera or per pool slot. Each process handles its frames, and results come back through **IPC** (pipes, shared memory or message queues) instead of shared variables.

Measured creation cost (2,000 repetitions, my sandbox):

create+join thread : 11.4 us

fork+wait process : 79.0 us

ratio : 6.9x

Your numbers will differ, but a process is consistently several times more expensive to create than a thread.

	Threads	**Processes**
Creation	about 11 µs	about 79 µs (7× more)
Context switch	cheaper: same address space, no TLB/page-table switch	dearer: address-space switch, TLB flush
Memory	share code, heap and globals; private stack only	separate address spaces (copy-on-write helps at first)
Combining results	shared variables plus a lock, no copying	pipes or shared memory, with kernel copying
Fault isolation	one crash or bad pointer kills all threads	one crash affects only that process
Synchronisation	mutexes needed for every shared update	little sharing, so fewer races

Complexity. For n cameras both approaches do O(n) work and are limited by cores. Memory is O(n·stack) for threads and O(n·address space) for processes. Result collection is O(1) per update with a lock for threads, versus O(data) copying per message for pipes.

Verdict. Threads are better for the plate reader (fast, light, easy to combine results). Processes would be chosen only if isolation matters, for example to survive a crashing plate-recognition library.

6. Commands for your screenshots

bash

```
nproc && lscpu | head -20
gcc -O1 -std=c11 -Wall -Wextra -pthread plates.c -o plates && ./plates
gcc -O0 -std=c11 -Wall -Wextra -pthread race.c  -o race  && ./race
gcc -O0 -std=c11 -Wall -Wextra -pthread race2.c -o race2 && ./race2
gcc -O1 -std=c11 -Wall -Wextra -pthread create.c -o create && ./create
git add . && git commit -m "Task03: threads"
```

Q4 – Inter-Process Communication **6 Marks**

Bloom's Level: K5 – Evaluate

1. Complete the Code: pipe (TODO 1)

The completed loop in pipe_readings.c:

c

```
while (read(fd[0], &r, sizeof r) == (ssize_t)sizeof r) {
    count++;
    sum += r;
    if (r > max) max = r;
}
```

read returns the number of bytes read. It returns 0 only when the pipe is empty and every write end is closed (end of data), so the loop stops cleanly at that point.

Output, matching the expected output:

analyser: 6 readings, average 35.5, max 41

reader: done

Remove close(fd[1]) in the parent: why the child never finishes

I added a -DNOCLOSE option to the file so you can test this. Compile with gcc -DNOCLOSE pipe_readings.c -o pipe_noclose and run it. In my run it hung and was killed by timeout (exit status 124).

Explanation.

- read returns 0 (end of data) only when all write ends of the pipe are closed.
- The parent still holds its write end fd[1], and the child also holds a copy of fd[1] inherited at fork. The child closed its copy at the start, but the parent's copy stays open.
- So after the 6 readings the pipe is empty, but the kernel sees a live writer and blocks the child's read forever, waiting for data that never comes.
- The parent meanwhile blocks in waitpid waiting for the child. Neither can proceed, so the program hangs.

That answers the first question in Q4: the pipe signals end of data to the reader through closing the write end, which makes read return 0.

2. Second IPC implementation: shared memory + semaphores (shm_readings.c)

It uses POSIX shared memory (shm_open + mmap) holding a one-slot buffer and two semaphores created with pshared = 1:

- empty (starts at 1) means the slot is free.
- full (starts at 0) means a reading is ready.

The reader waits on empty, writes the value, and posts full. The analyser does the reverse. A done flag replaces closing the pipe as the end-of-data signal.

analyser: 6 readings, average 35.5, max 41

reader: done

Compile with gcc -std=c11 -Wall -Wextra shm_readings.c -o shm_readings -lrt -pthread.

3. Timing: 1 MB, 10 MB and 100 MB (ipc_bench.c)

The program transfers data in 64 KB chunks between a parent and a child process, once through a pipe and once through shared memory with semaphores. Each size is repeated 5 times and the best time is kept. It reads one byte per cache line on the receiving side and compares checksums to prove both methods deliver the same data.

Compile and run: gcc -O2 -std=c11 -Wall -Wextra ipc_bench.c -o ipc_bench -lrt -pthread && ./ipc_bench

My sandbox output (run twice):

Size	Pipe	Shared memory + semaphore	Pipe / shm
1 MB	0.0004 s	0.0002 s	1.5×
10 MB	0.0021–0.0023 s	0.0009–0.0010 s	2.2×
100 MB	0.016–0.021 s	0.007–0.009 s	2.2×

Evaluation (confirm against your own numbers):

- Both methods scale about linearly with data size, so both are O(n) in the amount of data.
- Shared memory was roughly 2× faster once the data was large enough. At 1 MB the fixed costs (fork, setup) hide the difference.
- The gap is smaller than “pipe does two copies, shm does none” suggests, because in this benchmark the shared-memory version still does one memcpy into the shared region (the data has to come from somewhere), and the semaphores add a synchronisation round trip per chunk.
- Chunk size matters: larger chunks mean fewer system calls and semaphore operations. Try changing CHUNK and note the effect.

4. Answers to the “Determine” questions

Shared memory vs message passing.

- In shared memory, processes map the same region and read or write it directly. The kernel only sets it up, so it is fast but the programmer must synchronise access.
- In message passing (pipes, message queues, sockets), processes exchange messages through the kernel with send/receive-style calls. It is easier and safer, and it works across machines, but each message costs a system call and a copy.

Which copies data.

- A pipe or message queue copies the data twice: user buffer → kernel buffer → receiver’s user buffer.
- Shared memory has no kernel copy, because both processes access the same physical pages. (The producer still has to put the data there once, which my benchmark includes.)

Which needs extra synchronisation, and why.

- Shared memory needs it. Nothing stops the reader from overwriting a value the analyser hasn’t consumed, or the analyser from reading a half-written value. That is why shm_readings.c needs the empty/full semaphores.
- A pipe synchronises for you: read blocks when the pipe is empty, and write blocks when it is full.

5. Alternative concept and complexity

	Pipe	Shared memory + semaphore
Time to transfer n readings	O(n) with two kernel copies, plus a system call per read/write	O(n) with no kernel copy. The only system calls are semaphore operations when a process has to block.
Extra synchronisation	none needed, built in	required: semaphores or mutex
Space	kernel buffer (64 KB by default on Linux)	one shared region sized by you, O(buffer size)
End of data	close write end, read returns 0	flag or sentinel value

	Pipe	Shared memory + semaphore
Risk	a leaked write end causes a hang (shown above)	a missing semaphore post or wait gives a race or deadlock

Justified choice. For this weather station, which sends small, steady readings and must not lose any, the **pipe** is the better choice: simple, safe and fast enough. For bulk or high-rate data (like the 100 MB case), **shared memory with a ring buffer and semaphores** is better, as long as the synchronisation is written carefully.

6. Commands for your screenshots

bash

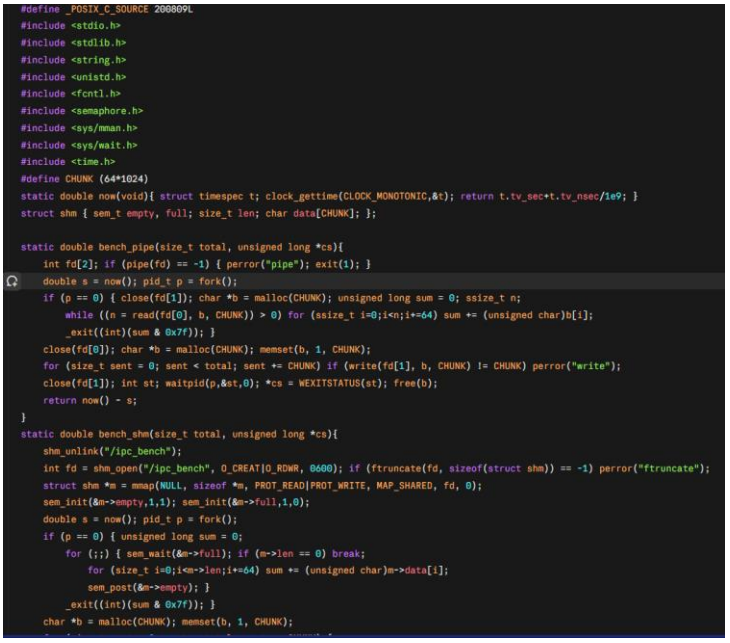
gcc -std=c11 -Wall -Wextra pipe_readings.c -o pipe_ok && ./pipe_ok

gcc -std=c11 -Wall -Wextra -DNOCLOSE pipe_readings.c -o pipe_noclose && timeout 3 ./pipe_noclose; echo "exit: $?"

gcc -std=c11 -Wall -Wextra shm_readings.c -o shm_readings -lrt -pthread && ./shm_readings

gcc -O2 -std=c11 -Wall -Wextra ipc_bench.c -o ipc_bench -lrt -pthread && ./ipc_bench

git add . && git commit -m "Task04: IPC pip



```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <time.h>
#define CHUNK (64*1024)
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
struct shm { sem_t empty, full; size_t len; char data[CHUNK]; };

static double bench_pipe(size_t total, unsigned long *cs){
    int fd[2]; if (pipe(fd) == -1) { perror("pipe"); exit(1); }
    double s = now(); pid_t p = fork();
    if (p == 0) { close(fd[1]); char *b = malloc(CHUNK); unsigned long sum = 0; ssize_t n;
        while ((n = read(fd[0], b, CHUNK)) > 0) for (ssize_t i=0;i<n;i+=64) sum += (unsigned char)b[i];
        _exit((int)(sum & 0x7f)); }
    close(fd[0]); char *b = malloc(CHUNK); memset(b, 1, CHUNK);
    for (size_t sent = 0; sent < total; sent += CHUNK) if (write(fd[1], b, CHUNK) != CHUNK) perror("write");
    close(fd[1]); int st; waitpid(p,&st,0); *cs = WEXITSTATUS(st); free(b);
    return now() - s;
}
static double bench_shm(size_t total, unsigned long *cs){
    shm_unlink("/ipc_bench");
    int fd = shm_open("/ipc_bench", O_CREAT|O_RDWR, 0600); if (ftruncate(fd, sizeof(struct shm)) == -1) perror("ftruncate");
    struct shm *m = mmap(NULL, sizeof *m, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
    sem_init(&m->empty,1,1); sem_init(&m->full,1,0);
    double s = now(); pid_t p = fork();
    if (p == 0) { unsigned long sum = 0;
        for (;;) { sem_wait(&m->full); if (m->len == 0) break;
            for (size_t i=0;i<m->len;i+=64) sum += (unsigned char)m->data[i];
            sem_post(&m->empty); }
        _exit((int)(sum & 0x7f)); }
    char *b = malloc(CHUNK); memset(b, 1, CHUNK);
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41 reader:
done
```

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors 11 Marks

Bloom's Level: K5 – Evaluate

1. Design from the principle (theory part)

The three requirements. A critical-section solution must give:

- **Mutual exclusion:** at most one process in the critical section at a time.

- **Progress:** if no one is inside, a process that wants in must not be postponed forever by processes that don't want in.
- **Bounded waiting:** once a process has asked to enter, only a limited number of others can enter before it.

Peterson's solution (two processes, i and j).

c

flag[i] = true; turn = j;

while (flag[j] && turn == j) ; /* wait */

/* critical section */

flag[i] = false;

- *Mutual exclusion:* both can be inside only if flag[i] and flag[j] are true and both passed the wait. But turn holds a single value (i or j), so the loop condition is false for only one of them.
- *Progress:* a process that doesn't want in has flag false, so the other isn't blocked.
- *Bounded waiting:* after process j leaves and wants in again, it sets turn = i, which gives i the next entry. So i waits for at most one turn of j.
- *Why it may fail on modern processors:* CPUs and compilers reorder memory operations. A store to flag[i] can become visible after the load of flag[j] (store buffers), so both processes can enter. It only works with memory barriers or sequentially consistent atomics. That's why real systems use hardware atomic instructions (test-and-set, compare-and-swap) and locks instead.

Semaphores. A semaphore is an integer S with two atomic operations: wait(S) (while S ≤ 0 block; S--) and signal(S) (S++, wake a waiter).

- **Binary semaphore (mutex, value 0 or 1)** protects the seat counter: wait(mutex); if (seats > 0) seats--; signal(mutex);. Only one agent updates at a time, so no seat is sold twice.
- **Counting semaphore** initialised to the number of free seats can limit entry directly: wait(seats_sem) succeeds for exactly as many agents as there are seats.

Classic problems with semaphores

Bounded buffer (producer–consumer): mutex = 1, empty = N, full = 0.

- Producer: wait(empty); wait(mutex); add item; signal(mutex); signal(full)
- Consumer: wait(full); wait(mutex); remove item; signal(mutex); signal(empty)
- Swapping the two waits (taking mutex before empty) can deadlock, because a producer could hold the mutex while waiting for space that only a consumer (blocked on the mutex) can free.

Readers–writers (readers' priority): rw = 1, mutex = 1, readcount = 0.

- Writer: wait(rw); write; signal(rw)
- Reader: wait(mutex); readcount++; if (readcount==1) wait(rw); signal(mutex); read; wait(mutex); readcount--; if (readcount==0) signal(rw); signal(mutex)
- Many readers run together; a writer is exclusive. Writers can starve under steady reader traffic. Mention this as a weakness.

Dining philosophers (5): one semaphore per chopstick. The naive "pick left, then right" can deadlock if all pick left at once. Fixes:

- allow at most 4 philosophers at the table (an extra counting semaphore initialised to 4);
- pick up both chopsticks or neither, inside a critical section;
- an asymmetric rule (odd philosophers pick left first, even pick right first).

Monitors. A monitor bundles shared data with the procedures that access it, and only one process can be active inside at a time (mutual exclusion is automatic). Condition variables (wait, signal) let a

process sleep inside the monitor until a condition holds. For the booking system: book() does while (seats == 0) wait(not_empty); seats--;. This is easier to get right than raw semaphores, since no one can forget a wait/signal or get their order wrong. In C, a monitor is built from a mutex plus pthread_cond_t.

2. Complete the Code: TODO 1 and TODO 2 (bb.c)

Producer (TODO 1):

c

```
sem_wait(&empty);
pthread_mutex_lock(&lock);
buffer[in] = item;
in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&full);
```

Consumer (TODO 2):

c

```
sem_wait(&full);
pthread_mutex_lock(&lock);
int item = buffer[out];
out = (out + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&empty);
consumed_sum += item;
```

Output, matching the expected output:

items consumed: 20, sum = 210 (expected 210)

100 runs. for i in $(seq 100); do ./bb; done | sort | uniq -c gave:

100 items consumed: 20, sum = 210 (expected 210)

Note that consumed_sum += item is outside the mutex, as the TODO order says. That is safe only because there is a single consumer. With several consumers it would be a race, so in my multi-thread test I count consumed items inside the critical section.

3. Test with 1 and 4 producers/consumers, and without the mutex (bb_multi.c)

Each producer makes 5,000 unique items. Consumers count how many times each item is consumed. Run with ./bb_multi 1 and ./bb_multi 4.

1 producer(s), 1 consumer(s): 5000 items, consumed once=5000, never=0, more than once=0 -> CORRECT

4 producer(s), 4 consumer(s): 20000 items, consumed once=20000, never=0, more than once=0 -> CORRECT

Every item is consumed exactly once.

Without the mutex (-DNOMUTEX; it also yields occasionally to widen the race window, which I'd mention in your report):

1 producer(s), 1 consumer(s): ... consumed once=5000 -> CORRECT

4 producer(s), 4 consumer(s): 20000 items, consumed once=9827, never=5628, more than once=4545 -> WRONG

Evaluation. With one producer and one consumer, the semaphores alone are enough, because only one thread touches in and one touches out. With several, two producers can read the same in and

write the same slot (lost items), and two consumers can read the same slot (duplicates). The mutex is needed to protect in, out and the buffer slots.

4. Deployment decision: lock granularity (booking.c)

16 agent threads each make 400 bookings over 8 trains × 10 coaches × 40 seats, with a 100 µs delay inside the critical section to model database or payment work. The three versions use one lock for the whole system, one per train, and one per coach. It also checks that no seat is sold twice.

Locking	Time	Bookings/s	Double-sold seats
Whole system	1.07 s	about 6,000	0
Per train	0.17 s	about 36,700	0
Per coach	0.08 s	about 81,500	0

All three sell the same 2,761 seats, so all are correct. They differ only in throughput.

Choice: one lock per coach. It is correct (a seat belongs to exactly one coach) and about 14× faster than a global lock here. The extra cost is more locks (80 in this test) and more care with operations spanning coaches. For a booking that needs seats in several coaches, take the locks in a fixed order (for example by coach number) to avoid deadlock.

Caveat for your report. The speed-up comes largely from the simulated delay inside the lock. With real, very short critical sections on a multi-core machine, the gap would be different, so quote your own run.

5. Alternative concept: atomics and spin locks (lockbench.c)

It performs 4,000,000 counter updates in total, split over 1, 4 and 16 threads, using atomic_fetch_add, a spin lock (atomic_flag) and a mutex. CPU/wall is CPU time divided by wall time; it shows how much CPU the threads burn.

Method	Threads	ns/update	CPU/wall
atomic_fetch_add	1 / 4 / 16	6.1 / 5.9 / 6.2	about 1.0
spin lock	1 / 4 / 16	7.4 / 17.4 / 69.3	about 1.0
mutex	1 / 4 / 16	17.4 / 17.3 / 16.2	0.95–0.99

(My machine has 1 core, so the CPU/wall ratio stays near 1 for all of them. On your multi-core machine, the spin lock's CPU/wall should rise toward the thread count, which is the wasted spinning. Report your own values.)

Evaluation.

- **Atomics:** the cheapest and the best for a simple shared counter. There is no lock, no blocking and no spinning wasted.
- **Spin lock:** good only for very short critical sections with few threads on multiple cores. Waiting threads burn CPU. On my 1-core machine, a spinner burns its whole time slice while the lock holder can't run, which is why the cost grows to 69 ns at 16 threads.
- **Mutex:** the cost is steady, and waiting threads sleep and give up the CPU. That is slower per update but wastes no CPU.
- **Long critical section:** use a mutex or semaphore, never a spin lock. In the booking example, with a 100 µs delay, spinners would burn CPU the whole time.

Complexity. Each update is O(1) for all three. Under contention with n threads, a spin lock wastes up to O(n) CPU per update (n−1 spinners), while a mutex wastes none, but pays system call and context-switch cost when it has to sleep. Space is O(1) for the counter and the lock.

6. Commands for your screenshots

bash

gcc -std=c11 -Wall -Wextra -pthread bb.c -o bb && ./bb

for i in $(seq 100); do ./bb; done | sort | uniq -c

gcc -std=c11 -Wall -Wextra -pthread bb_multi.c -o bb_multi && ./bb_multi 1 && ./bb_multi 4

gcc -std=c11 -Wall -Wextra -pthread -DNOMUTEX bb_multi.c -o bb_nomutex && ./bb_nomutex 4

gcc -O2 -std=c11 -Wall -Wextra -pthread booking.c -o booking && ./booking

gcc -O2 -std=c11 -Wall -Wextra -pthread lockbench.c -o lockbench && ./lockbench

git add . && git commit -m "Task05: synchronisation"

bb_nomutex 4 can occasionally hang or give odd results, which is itself evidence of the broken synchronisation. If it runs too long, stop it with Ctrl+C and screenshot what you have.

For the report, also draw the semaphore pseudocode for readers–writers and dining philosophers (section 1) in your own words. The viva may ask you to explain them.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm 13 Marks

Bloom's Level: K5 – Evaluate

A. Characterise Deadlock (2 Marks)

1. Four necessary conditions for deadlock

1. Mutual Exclusion: Only one process can use a resource at a time.
2. Hold and Wait: A process holds one resource while waiting for another.
3. No Preemption: Resources cannot be forcibly taken from a process.
4. Circular Wait: Processes wait for resources held by one another in a circular chain.

Resource Allocation Graph example

Example:

- P1 holds GPU and waits for a licence.
- P2 holds the licence and waits for storage.
- P3 holds storage and waits for GPU.

This creates a circular wait and can lead to deadlock.

2. Completed C Program – Banker's Algorithm

Copy this program into your C compiler.

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};

int maxm[P][R] = {
```

```
    {7, 5, 3}, {3, 2, 2}, {9, 0, 2},
    {2, 2, 2}, {4, 3, 3}
};

int alloc[P][R] = {
    {0, 1, 0}, {2, 0, 0}, {3, 0, 2},
    {2, 1, 1}, {0, 0, 2}
};

int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);

    while (count < P) {
        int found = 0;

        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;

            int ok = 1;

            for (int r = 0; r < R; r++) {
                if (maxm[p][r] - alloc[p][r] > work[r])
                    ok = 0;
            }

            if (ok) {
                for (int r = 0; r < R; r++)
                    work[r] += alloc[p][r];

                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
        }

        if (!found) return 0;
    }
```

```
    return 1;
}

void request(int p, int req[R])
{
    for (int r = 0; r < R; r++) {
        if (req[r] > maxm[p][r] - alloc[p][r] ||
            req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    }

    for (int r = 0; r < R; r++) {
        avail[r] -= req[r];
        alloc[p][r] += req[r];
    }

    int seq[P];

    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) {
            avail[r] += req[r];
            alloc[p][r] -= req[r];
        }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];

    if (is_safe(seq)) {
        printf("initial state safe:");
```

```
        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);
        printf("\n");
    }

    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});

    return 0;
}
```

Output:

initial state safe: P1 P3 P4 P0 P2

P1 request: granted, safe sequence P1 P3 P4 P0 P2

P4 request: must wait (not available)

P0 request: denied (unsafe)

Explanation:

- P1 request is granted because the system remains in a safe state.
- P4 request must wait because the available resources are insufficient.
- P0 request is denied because granting it makes the system unsafe.

B. Compare the Strategies (3 Marks)

Strategy	**Explanation**	**Advantage / Disadvantage**
Prevention	Removes one of the four deadlock conditions.	Prevents deadlock but reduces resource flexibility.
Avoidance	Uses Banker's algorithm before granting requests.	Keeps the system in a safe state but requires maximum resource information.
Detection and Recovery	Detects deadlock and terminates or restarts processes.	Allows resource sharing but may lose completed work.
Ostrich Approach	Ignores deadlocks.	Simple, but system failure may occur.

Evaluation:

For the research centre, deadlock avoidance is suitable because GPUs, software licences and storage are expensive shared resources. Banker's algorithm checks whether resource allocation keeps the system safe.

However, detection and recovery can be useful when maximum resource requirements are not known in advance.

C. Apply the Banker's Algorithm (4 Marks)

1. Given Available Resources

Resource Available

GPU	3
Licence	3
Storage	2

2. Need Matrix

Formula:

$$ Need = Max - Allocation $$

Process	GPU	Licence	Storage
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

3. Safe Sequence

The safe sequence is:

$$ \boxed{P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2} $$

This means all five processes can complete without deadlock.

4. Resource Request Results

Request	Result	Reason
P1 (1,0,2)	Granted	System remains safe.
P4 (3,3,0)	Wait	Resources are unavailable.
P0 (0,2,0)	Denied	System becomes unsafe.

Why is P0's request unsafe?

Before the request, available resources are (2,3,0), after granting P1's request.

After temporarily granting P0's request (0,2,0):

$$ Available=(2,1,0) $$

P0 still needs (7,2,3), which is greater than the available resources.

No process can complete with the remaining available resources. Therefore, the system enters an unsafe state, and P0's request is denied.

D. Predict Before Implementing (1 Mark)

The running time of the Banker’s algorithm depends on:

- n = Number of processes.
- m = Number of resource types.

The time complexity is:

$$ \boxed{O(m \times n^2)} $$

Explanation:

The algorithm checks each process and compares its resource needs with available resources. It may repeat the checking process until all processes finish.

Therefore:

- Increasing the number of processes increases execution time.
- Increasing resource types also increases execution time.
- The algorithm becomes slower when the system has many processes.

E. Measure (1 Mark)

Measure the execution time for different numbers of processes.

Number of processes	Expected behaviour
10	Very fast
100	More execution time
1,000	Higher execution time

Use the C clock() function to measure execution time.

```
#include <stdio.h>
#include <time.h>

int main()
{
   clock_t start, end;

   start = clock();

   // Call the safety algorithm here

   end = clock();

   printf("Time = %f seconds\n",
   (double)(end - start) / CLOCKS_PER_SEC);

   return 0;
}
```

Observation:

As the number of processes increases, the execution time also increases. This supports the predicted complexity of $O(m \times n^2)$.

Note: Run the complete safety algorithm with 10, 100 and 1,000 processes to obtain actual measurements. Do not enter estimated times as measured results.

F. Defend the Final Design (2 Marks)

Recommended Strategy: Deadlock Avoidance using Banker's Algorithm

For the research centre, I recommend Banker's algorithm because it checks resource availability before granting requests.

Reasons:

1. It prevents the system from entering an unsafe state.

2. It avoids deadlocks by checking resource requirements.
3. It improves the reliability of shared GPU, licence and storage resources.
4. It allows resources to be allocated safely among multiple processes.

Limitations:

- Maximum resource requirements must be known in advance.
- Safety checking increases execution time.
- It may delay some processes while waiting for resources.

Final Evaluation:

Banker's algorithm is suitable for the research centre because it provides safe resource allocation and reduces the possibility of deadlocks. Although it requires additional checking time, it improves overall system reliability and prevents unnecessary process failures.

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available) P0 request: denied (unsafe)
```

.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6
Question	**Assessed Area**	**Bloom's Level**	**Marks**
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6

- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

Model → Design → Implement → Measure → Compare → Evaluate → Recommend